Design Patterns in Java

The Art of Thinking Like an Architect

When you've written Java for a few years, you start noticing something:
you keep solving the same kind of problems — object creation, request handling, class dependency, communication between modules — again and again.

Design Patterns are **battle-tested templates** for solving these recurring problems elegantly.

I learned this the hard way during a fintech project.
We had multiple classes reading payment data from different sources — CSV, XML, and APIs.
Every time a new payment type came in, someone would copy-paste another "reader" class with almost the same code.
Then one senior architect rewrote it using the **Factory Pattern**.
Suddenly, adding a new data source took 5 lines instead of 50.
That's when it clicked —

"Design Patterns aren't fancy theory — they're shortcuts to building maintainable, scalable systems."

In interviews, design patterns are where you prove you can *design* — not just *code*.
You'll often get questions like:

- "How does Spring use the Singleton pattern?"
- "Where have you applied Strategy or Observer in your projects?"
- "Can you implement a thread-safe Singleton?"

This chapter will make sure you can answer those — confidently, and with real examples.

What You'll Master in This Chapter

- **Creational Patterns**

  → Controlling object creation efficiently.

  - Singleton
  - Factory
  - Builder
- **Structural Patterns**

  → How classes and objects connect.

  - Adapter
  - Decorator
  - Proxy
- **Behavioral Patterns**

  → How objects communicate and behave.

  - Strategy
  - Observer
  - Command
- **MVC Pattern and Dependency Injection Basics**

  → How modern frameworks (like Spring MVC) are structured using these patterns internally.

Famous Free Documents & Resources

1. **Design Pattern Fundamentals**
    - Refactoring Guru – Design Patterns in Java → One of the best interactive resources.
    - Baeldung – Design Patterns in Java → Practical examples.
    - GeeksforGeeks – Design Patterns in Java
2. **Creational Patterns**
    - Baeldung – Singleton Pattern
    - Baeldung – Factory Pattern
    - GeeksforGeeks – Builder Pattern
3. **Structural Patterns**
    - Baeldung – Proxy Pattern
    - Baeldung – Decorator Pattern
    - GeeksforGeeks – Adapter Pattern
4. **Behavioral Patterns**
    - Baeldung – Strategy Pattern
    - Baeldung – Observer Pattern
    - GeeksforGeeks – Command Pattern

Expert Notes & Gotchas

- **Singleton is everywhere — but use it carefully.**
  It ensures a single instance across the app, but it can easily break testability and scalability.
  Spring's beans are Singletons *by default*, but managed safely by the IoC container.
- **Factory Pattern is the backbone of frameworks.**
  Spring uses factories to create beans dynamically based on configuration.
  It's the reason you can change an implementation without touching the client code.
- **Builder Pattern = readability.**
  Use it when constructors have too many parameters.
  You'll see it in libraries like Lombok @Builder or StringBuilder itself.
- **Decorator Pattern adds functionality dynamically.**
  Spring's HandlerInterceptor and Servlet filters use this concept to add behaviors like logging, auth, or caching.
- **Proxy Pattern powers Spring AOP.**
  When you annotate a method with @Transactional or @Cacheable, Spring wraps your class with a proxy to inject behavior before/after method calls.
- **Strategy Pattern = flexibility.**
  Example: Different payment gateways (Razorpay, PayPal, Stripe) — each implements a common interface, selected at runtime.
- **Observer Pattern drives real-time systems.**
  You'll see it in event-driven architectures, pub-sub models, and even UI updates.
- **Command Pattern = decoupled requests.**
  Used in job schedulers, undo-redo systems, and distributed task queues.
- **MVC isn't just about UI.**
  In Spring MVC,
  - Model = Data layer (Entities/DTOs)
  - View = API Response (JSON/XML)
  - Controller = Business logic and routing layer.

Example 1: Thread-Safe Singleton

```
public class ConfigManager {
    private static volatile ConfigManager instance;
    private ConfigManager() {}

    public static ConfigManager getInstance() {
        if (instance == null) {
            synchronized (ConfigManager.class) {
                if (instance == null)
                    instance = new ConfigManager();
            }
        }
        return instance;
    }
}
```

Uses **double-checked locking** — ensures thread safety and performance.

Example 2: Factory Pattern for Payment Gateways

```
interface PaymentGateway {
    void processPayment(double amount);
}

class Razorpay implements PaymentGateway {
    public void processPayment(double amount) {
        System.out.println("Processing via Razorpay: " + amount);
    }
}

class PayPal implements PaymentGateway {
    public void processPayment(double amount) {
        System.out.println("Processing via PayPal: " + amount);
    }
}

class PaymentFactory {
    public static PaymentGateway getGateway(String type) {
        return switch (type.toLowerCase()) {
            case "paypal" -> new PayPal();
            default -> new Razorpay();
        };
    }
}
```

💡 This is real-world stuff — change your payment processor without changing your main code.

Example 3: Strategy Pattern for Sorting Algorithms

```
interface SortStrategy {
    void sort(List<Integer> numbers);
}

class QuickSort implements SortStrategy {
    public void sort(List<Integer> numbers) { System.out.println("QuickSort"); }
}

class MergeSort implements SortStrategy {
    public void sort(List<Integer> numbers) { System.out.println("MergeSort"); }
}

class SortContext {
    private SortStrategy strategy;
    public SortContext(SortStrategy strategy) { this.strategy = strategy; }
    public void executeSort(List<Integer> numbers) { strategy.sort(numbers); }
}
```

Plug and play new algorithms — clean, testable, and flexible.

Once you understand design patterns, you'll stop copying code and start **designing systems**.

You'll see patterns everywhere — in Spring, in microservices, even in caching and messaging systems.

Question Categorization Overview

To make this guide more practical for different experience levels and roles, I've categorized every question in this chapter based on:

1. **Experience Level (L1–L4)** – representing how deep the understanding required is.
2. **Role Focus** – identifying which job profile or area the question is most relevant to.

This way, you can easily filter and prepare according to your **career stage** and **target role**.

Level	Experience Range	Description
L1	0–2 years	Foundational Java concepts; syntax, data types, control flow, OOP basics
L2	2–5 years	Intermediate backend engineer; memory, exceptions, collections, concurrency, and frameworks
L3	5–9 years	Senior engineer; architecture, performance, multithreading, and optimization
L4	9+ years	Tech Lead / Architect; deep JVM internals, design trade-offs, scalability, and tuning

Role	Focus Area
Backend Core	Core Java, Collections, Multithreading, Exception Handling
Full-Stack Developer	Java backend + some frontend integration (REST APIs, JSON, etc.)
Microservices Engineer	Spring Boot, REST, concurrency, distributed systems
System Design Engineer	Architecture, caching, GC tuning, concurrency
Data/Integration Engineer	File I/O, serialization, streaming, data handling
Security Engineer	Encryption, transient fields, safe serialization

How to Use This Categorization

- **If you're a beginner:** Focus on **L1 questions** – they form the foundation for every backend interview.
- **If you're preparing for mid-level roles (SDE-1/2):** Prioritize **L2 questions**, especially on collections, exceptions, and concurrency.
- **If you're moving toward senior roles:** Practice **L3–L4** – JVM tuning, concurrency patterns, and architectural decisions.
- **If you're targeting a specific role (like Microservices or Security):** Use the *Role Focus* column to zero in on your domain.

#	Question	Level	Target Role
1	What is the Singleton Design Pattern in Java?	L1	Java Developer, Backend Core
2	What are the advantages of using the Singleton Design Pattern?	L1	Java Developer, Backend Core
3	What is the Factory Design Pattern in Java?	L1	Java Developer, Backend Core
4	What are the benefits of the Factory Design Pattern?	L2	Backend Engineer, API Developer
5	What is the Builder Design Pattern in Java?	L2	Backend Engineer
6	What are the benefits of the Builder Design Pattern?	L2	Backend Engineer, Full-Stack Dev
7	What is the Adapter Design Pattern in Java?	L2	Backend Core, Integration Engineer
8	What are the benefits of the Adapter Design Pattern?	L2	Backend Core, Integration Engineer
9	What is the Decorator Design Pattern in Java?	L2	Backend Engineer, API Layer Dev
10	What are the advantages of the Decorator Design Pattern?	L2	Backend Engineer
11	What is the Proxy Design Pattern in Java?	L2	Backend Core, System Engineer
12	What are the benefits of the Proxy Design Pattern?	L2	Backend Core, System Engineer
13	What is the Strategy Design Pattern in Java?	L2	Backend Engineer, Product Dev
14	What are the advantages of the Strategy Design Pattern?	L2	Backend Engineer, Product Dev
15	What is the Observer Design Pattern in Java?	L2	Backend Engineer, Event-Driven Dev
16	What are the benefits of the Observer Design Pattern?	L2	Backend Engineer, Event-Driven Dev
17	What is the Command Design Pattern in Java?	L2	Backend Engineer, Systems Dev
18	What are the benefits of the Command Design Pattern?	L2	Backend Engineer, Systems Dev
19	What is the Model-View-Controller (MVC) Pattern in Java?	L1	Web Developer, Spring Dev
20	What are the advantages of the MVC Pattern?	L1	Web Developer, Spring Dev

21	How does Dependency Injection relate to Dependency Inversion?	L2	Backend Core, Spring Dev
22	What are the different types of Dependency Injection in Java?	L2	Backend Core, Spring Dev
23	What is the difference between Singleton Pattern and Dependency Injection?	L2	Backend Core, Spring Dev
24	What are the real-world use cases of the Strategy Design Pattern?	L3	Backend Engineer, Product Dev
25	How is the Observer Pattern used in Event Handling Systems?	L3	Backend Engineer, Event-Driven Dev
26	What is Lazy Initialization in Singleton? How is it implemented?	L2	Backend Engineer
27	How does the Decorator Pattern compare to Subclassing?	L3	Senior Backend Engineer
28	How is MVC used in Web Applications?	L1	Spring Dev, Web Engineer
29	How does the Proxy Pattern improve performance in large systems?	L3	Senior Backend, System Engineer
30	What is the role of DI in Testing?	L2	Backend Core, QA Automation
31	Difference between Factory Method and Abstract Factory	L3	Senior Backend Engineer
32	How does the Command Pattern handle Undo/Redo operations?	L3	Senior Backend Engineer
33	Advantages and disadvantages of the Observer Pattern	L3	Senior Backend Engineer
34	How can the Decorator Pattern improve logging functionality?	L3	Senior Backend Engineer
35	Role of MVC in a Spring Framework-based Web Application	L2	Spring Boot Dev
36	Real-world scenarios for the Proxy Design Pattern	L3	Senior Backend, Architect
37	How does the Builder Pattern simplify complex construction?	L2	Backend Engineer
38	Strategy Pattern in AI decision-making systems	L3	Backend Engineer, AI Systems Dev
39	Observer Pattern for real-time notifications	L3	Backend Engineer, Event-Driven Dev
40	What is Dependency Injection in Spring, and how is it implemented?	L2	Backend Core, Spring Boot Dev

41	What is the Prototype Design Pattern in Java, and when to use it?	L2	Backend Engineer
42	What is the Object Pool Pattern, and why is it useful?	L3	Senior Backend, Performance Engineer
43	What is the Composite Pattern, and where is it used?	L3	Senior Backend, Architect
44	What is the Facade Pattern, and how does it simplify subsystems?	L2	Backend Engineer, API Designer
45	How does the Flyweight Pattern improve memory efficiency?	L3	Senior Backend, Performance Engineer
46	What is the Bridge Pattern, and how does it differ from Adapter?	L3	Senior Backend, Architect
47	What is the Template Method Pattern in Java, and usage?	L3	Senior Backend, Framework Dev
48	What is the State Pattern, and how does it model behavior?	L3	Senior Backend, Workflow Engineer
49	What is the Mediator Pattern, and how does it simplify communication?	L3	Senior Backend, System Architect
50	What is the Chain of Responsibility Pattern, and where used?	L3	Senior Backend, Middleware Dev
51	What is the Interpreter Pattern, and real-world use cases?	L4	Senior Backend, Compiler/Rules Dev
52	What is the Visitor Pattern, and how does it help extend ops?	L4	Senior Backend, Compiler/Framework Dev
53	What is the Memento Pattern, and how does it enable undo?	L3	Senior Backend, System Dev
54	What is the Iterator Pattern, and its use in Java Collections?	L1	Java Developer, Backend Core

1. What is the Singleton Design Pattern in Java?

Answer

Singleton ensures exactly one instance of a class for the whole JVM and exposes a single way to access it. I'd use it for cross-cutting services like configuration, connection pools, or an application-wide metrics registry—things you don't want multiple copies of.

Common, safe implementations:

```
// Best: Enum (simple, thread-safe, resists reflection/serialization issues)
public enum AppConfig {
    INSTANCE;
    private final Properties props;
    AppConfig() {
        props = new Properties();
        // load props...
    }
    public String get(String key) { return props.getProperty(key); }
}
// Also good: Initialization-on-demand holder (lazy + thread-safe)
public class Logger {
    private Logger() {}
    private static class Holder { static final Logger INSTANCE = new Logger(); }
    public static Logger getInstance() { return Holder.INSTANCE; }
}
```

If I must use double-checked locking, I'll mark the instance volatile to avoid reorder issues:

```
public class Service {
    private static volatile Service instance;
    private Service() {}
    public static Service getInstance() {
        if (instance == null) {
            synchronized (Service.class) {
                if (instance == null) instance = new Service();
            }
        }
        return instance;
    }
}
```

Practical notes: guard against reflection (throw if instance already exists), handle serialization with a readResolve method, and keep the class focused (a bloated singleton becomes a hidden global).

2. What are the advantages of using the Singleton Design Pattern?

Answer

1. Single source of truth for shared resources
   When an e-commerce app needs the same tax rules or feature flags everywhere, a singleton centralizes that state and avoids “drift”.
2. Controlled, global access
   You get a well-known access point (e.g., Logger.getInstance()) without passing references through every constructor.
3. Performance and memory savings for heavy objects
   If initializing a pricing rules engine or a database driver is expensive, doing it once reduces startup and memory overhead.
4. Thread-safe coordination (when implemented correctly)
   One instance simplifies synchronization of shared concerns like caches or rate limiters.
5. Test seams via indirection
   Wrapping a dependency (e.g., a clock, config, or payment client) behind a singleton factory allows swapping test doubles in controlled environments.

Tiny example (centralized logger used across modules):

```
public enum AppLogger {
    INSTANCE;
    public void log(String msg) {
        System.out.println("[APP] " + msg);
    }
}

// anywhere
AppLogger.INSTANCE.log("Checkout started");
```

I still avoid overusing singletons; they’re great for infrastructure-level services, not for general business logic.

3. What is the Factory Design Pattern in Java?

Answer

Factory abstracts object creation so callers ask for “what” they need, not “how” it’s built. That keeps construction logic in one place and lets you add new variants without touching client code. I think of a payments module: based on method (UPI, Card, Wallet), the factory returns the right processor.

Simple factory:

```
interface PaymentProcessor { void pay(long paise); }

class UpiProcessor implements PaymentProcessor {
    public void pay(long paise) { /* UPI flow */ }
}
class CardProcessor implements PaymentProcessor {
    public void pay(long paise) { /* card flow */ }
}

class PaymentFactory {
    public static PaymentProcessor of(String method) {
        return switch (method.toLowerCase()) {
            case "upi" -> new UpiProcessor();
            case "card" -> new CardProcessor();
            default -> throw new IllegalArgumentException("Unsupported: " + method);
        };
    }
}

// usage
PaymentProcessor p = PaymentFactory.of("card");
p.pay(49900);
```

Factory method variant (subclasses decide what to create):

```
abstract class NotificationService {
    public void send(String msg) {
        Notifier n = createNotifier();
        n.push(msg);
    }
    protected abstract Notifier createNotifier();
}

class EmailService extends NotificationService {
    protected Notifier createNotifier() { return new EmailNotifier(); }
}
```

Why it helps: reduces conditionals scattered across the codebase, localizes creation policies (pooling, caching, config), and enables adding new product types with minimal client changes—useful when requirements evolve (new payment types, new message channels, new storage engines).

4. What are the benefits of the Factory Design Pattern?

Answer

The main value of the Factory pattern is that it separates the “what” from the “how” when it comes to object creation. A few key benefits:

1. Encapsulation of creation logic
   Instead of scattering if/else or new constructors across the code, the logic sits in one place. If the creation rules change, I only touch the factory.
2. Reusability
   A single factory can be reused across modules. For example, a NotificationFactory that produces SMS, Email, or Push objects can be shared by all services that need to notify users.
3. Flexibility for extensions
   Adding a new notification channel (say WhatsApp) means updating only the factory and creating the new class. Existing client code remains untouched.
4. Loose coupling
   Clients depend on abstractions (interfaces) rather than concrete classes, which aligns with the Dependency Inversion principle and makes the code easier to maintain and test.

Example with a messaging context:

```
interface Notification { void notifyUser(); }

class SMSNotification implements Notification {
    public void notifyUser() { System.out.println("Sending SMS..."); }
}
class EmailNotification implements Notification {
    public void notifyUser() { System.out.println("Sending Email..."); }
}

class NotificationFactory {
    public Notification create(String type) {
        return switch (type.toLowerCase()) {
            case "sms" -> new SMSNotification();
            case "email" -> new EmailNotification();
            default -> throw new IllegalArgumentException("Unsupported: " + type);
        };
    }
}
```

Client code doesn’t know or care how the object is created; it just uses the interface.

5. What is the Builder Design Pattern in Java?

Answer

The Builder pattern is about constructing complex objects step by step, without exposing the details of construction. It's especially useful when an object has many optional attributes or when you want to avoid constructors with long argument lists.

Take a Computer object with CPU, RAM, storage, GPU, etc. Some are required, some optional. Using Builder, the client can selectively set what's needed in a fluent way.

```
class Computer {
    private final String cpu;
    private final String ram;
    private final String storage;

    private Computer(Builder b) {
        this.cpu = b.cpu;
        this.ram = b.ram;
        this.storage = b.storage;
    }

    static class Builder {
        private String cpu;
        private String ram;
        private String storage;

        public Builder cpu(String cpu) { this.cpu = cpu; return this; }
        public Builder ram(String ram) { this.ram = ram; return this; }
        public Builder storage(String storage) { this.storage = storage; return this; }

        public Computer build() { return new Computer(this); }
    }
}

// usage
Computer laptop = new Computer.Builder()
    .cpu("M3 Pro")
    .ram("32GB")
    .storage("1TB SSD")
    .build();
```

This avoids telescoping constructors and improves readability. It also makes immutability straightforward since all fields can be declared final.

6. What are the benefits of the Builder Design Pattern?

Answer

1. Handles complexity

   When an object has many fields (some required, some optional), Builder makes it easy to construct without passing dozens of arguments in the constructor.
2. Fluent and readable

   The chained method style is intuitive and documents itself. Reading new Car.CarBuilder().setColor("Red").setSunroof(true).build() is much clearer than a constructor with 5–6 parameters.
3. Immutability

   Because the object is created in one shot after all values are set in the builder, the target object can be declared immutable.
4. Separation of concerns

   The builder class handles the process of construction, while the main class focuses on representing the data.

Example in a car booking system:

```
class Car {
    private final String engine;
    private final String color;
    private final boolean sunroof;

    private Car(CarBuilder b) {
        this.engine = b.engine;
        this.color = b.color;
        this.sunroof = b.sunroof;
    }

    static class CarBuilder {
        private String engine;
        private String color;
        private boolean sunroof;

        public CarBuilder engine(String engine) { this.engine = engine; return this; }
        public CarBuilder color(String color) { this.color = color; return this; }
        public CarBuilder sunroof(boolean sunroof) { this.sunroof = sunroof; return
this; }
```

```
        public Car build() { return new Car(this); }
    }
}

// usage
Car sedan = new Car.CarBuilder().engine("V8").color("Blue").sunroof(false).build();
```

So, the Builder pattern shines wherever clarity, immutability, and scalability in object construction are needed.

7. What is the Adapter Design Pattern in Java?

Answer

The Adapter pattern is about making two incompatible interfaces work together without changing their code. Think of it like a power plug adapter—your laptop has a 3-pin charger, but the socket only takes 2-pins. The adapter converts one interface into another.

In Java, we often use it when we want to integrate legacy code with new systems or when we need to reuse an external library that doesn't match our existing interface.

Example with a media player:

```
// Target interface
interface MediaPlayer {
    void play(String type, String file);
}

// Adaptee
class AdvancedMediaPlayer {
    void playMp4(String file) { System.out.println("Playing mp4: " + file); }
    void playVlc(String file) { System.out.println("Playing vlc: " + file); }
}

// Adapter
class MediaAdapter implements MediaPlayer {
    private final AdvancedMediaPlayer advanced = new AdvancedMediaPlayer();

    @Override
    public void play(String type, String file) {
        if (type.equalsIgnoreCase("mp4")) advanced.playMp4(file);
        else if (type.equalsIgnoreCase("vlc")) advanced.playVlc(file);
        else throw new IllegalArgumentException("Unsupported type: " + type);
    }
}

// Client
public class Main {
    public static void main(String[] args) {
        MediaPlayer player = new MediaAdapter();
        player.play("mp4", "movie.mp4");
    }
}
```

So, instead of changing either MediaPlayer or AdvancedMediaPlayer, the adapter glues them together.

8. What are the benefits of the Adapter Design Pattern?

Answer

1. Interoperability

   It lets systems with incompatible interfaces work together. For example, old code expecting processPayment() can still be used even if the new library provides executeTransaction().
2. Reusability

   Existing classes can be reused without modification, which is critical when we don't control third-party libraries.
3. Flexibility

   You can easily swap or extend adapters. For instance, in a payment gateway system, if you integrate Stripe tomorrow, you just add a new adapter.
4. Cleaner maintenance

   The conversion logic lives inside the adapter, so the rest of the codebase remains clean and decoupled.

Example in a payment system:

```
// Legacy interface
interface OldPaymentProcessor { void processPayment(double amt); }

// New library
class NewPaymentProcessor {
    void executeTransaction(double amt) {
        System.out.println("Processed: " + amt);
    }
}

// Adapter
class PaymentAdapter implements OldPaymentProcessor {
    private final NewPaymentProcessor newProcessor = new
NewPaymentProcessor();
    public void processPayment(double amt) {
        newProcessor.executeTransaction(amt);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
```

```
        OldPaymentProcessor processor = new PaymentAdapter();
        processor.processPayment(250.0);
    }
}
```

This way, legacy code continues working with the old interface while leveraging the new processor under the hood.

9. What is the Decorator Design Pattern in Java?

Answer

Decorator lets you add new behavior to objects dynamically without modifying their class. It's like ordering a basic coffee and then customizing it with milk, sugar, or caramel—each topping wraps the original object with extra behavior.

Unlike inheritance, which adds functionality at the class level, decorators add functionality at the object level using composition.

Example with a coffee shop:

```
// Component
interface Coffee {
    String getDescription();
    double getCost();
}

// Concrete component
class BasicCoffee implements Coffee {
    public String getDescription() { return "Basic Coffee"; }
    public double getCost() { return 2.0; }
}

// Base decorator
abstract class CoffeeDecorator implements Coffee {
    protected Coffee decoratedCoffee;
    public CoffeeDecorator(Coffee coffee) { this.decoratedCoffee = coffee; }
    public String getDescription() { return decoratedCoffee.getDescription(); }
    public double getCost() { return decoratedCoffee.getCost(); }
}

// Concrete decorator
class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) { super(coffee); }
    public String getDescription() { return super.getDescription() + ", Milk"; }
    public double getCost() { return super.getCost() + 0.5; }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Coffee order = new BasicCoffee();
        order = new MilkDecorator(order); // add milk dynamically
        System.out.println(order.getDescription() + " costs $" + order.getCost());
    }
```

```
}
```

Here, we didn’t modify BasicCoffee. We just wrapped it at runtime with decorators, making the system open to extension but closed to modification.

10. What are the advantages of the Decorator Design Pattern?

Answer

The key advantage of the Decorator is that it extends behavior dynamically without touching existing code. A few points:

1. Extensibility

   You can add new features by just writing a new decorator. No need to modify the base class.
2. Flexibility at runtime

   Decorators can be stacked in different combinations, so a single object can behave differently in different contexts.
3. Composability

   Since each decorator wraps another, you can build complex functionality piece by piece (like adding milk, then caramel, then whipped cream to a coffee).
4. Open/Closed Principle

   Your classes remain closed for modification but open for extension—meaning you avoid breaking stable code.

Example with coffee:

```
class CaramelDecorator extends CoffeeDecorator {
    public CaramelDecorator(Coffee coffee) { super(coffee); }
    public String getDescription() { return super.getDescription() + ", Caramel"; }
    public double getCost() { return super.getCost() + 0.7; }
}

// usage
public class Main {
    public static void main(String[] args) {
        Coffee order = new BasicCoffee();
        order = new MilkDecorator(order);
        order = new CaramelDecorator(order);  // layer decorators
        System.out.println(order.getDescription() + " costs $" + order.getCost());
    }
}
```

Here, you didn’t need multiple subclasses like CaramelMilkCoffee. You compose behavior at runtime.

11. What is the Proxy Design Pattern in Java?

Answer

Proxy provides a substitute or placeholder for another object to control access to it. The idea is: instead of talking directly to the heavy or sensitive object, you talk to a proxy that decides when and how to forward the request.

Common types:

- Virtual Proxy → delay creating expensive objects until needed (lazy loading)
- Protection Proxy → add access control (roles, permissions)
- Remote Proxy → represent an object located on a remote server

Example with images (virtual proxy):

```
interface Image { void display(); }

class RealImage implements Image {
    private String file;
    RealImage(String file) {
        this.file = file;
        loadFromDisk();
    }
    private void loadFromDisk() { System.out.println("Loading " + file); }
    public void display() { System.out.println("Displaying " + file); }
}

class ProxyImage implements Image {
    private RealImage real;
    private String file;
    ProxyImage(String file) { this.file = file; }
    public void display() {
        if (real == null) real = new RealImage(file); // lazy loading
        real.display();
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        Image img = new ProxyImage("photo.jpg");
        img.display(); // loads + displays
        img.display(); // just displays
    }
}
```

So, Proxy acts like a gatekeeper or middle layer, offering control without changing the real object.

12. What are the benefits of the Proxy Design Pattern?

Answer

1. Controlled Access

   Proxy can decide who can or cannot call the real object—useful in systems like banking or admin consoles.
2. Lazy Initialization

   Resource-heavy objects (images, reports, connections) are created only when first accessed, saving performance.
3. Performance Optimization

   A proxy can cache results or batch calls, reducing load on the actual object.
4. Security and Logging

   You can wrap extra concerns like authentication, authorization, or logging without polluting the real object's code.

Example in banking with a protection proxy:

```
interface BankAccount { void accessAccount(); }

class RealBankAccount implements BankAccount {
    public void accessAccount() {
        System.out.println("Accessing bank account details");
    }
}

class BankAccountProxy implements BankAccount {
    private RealBankAccount real;
    private String role;
    BankAccountProxy(String role) { this.role = role; }

    public void accessAccount() {
        if ("Admin".equalsIgnoreCase(role)) {
            if (real == null) real = new RealBankAccount();
            real.accessAccount();
        } else {
            System.out.println("Access Denied");
        }
    }
}

// usage
public class Main {
```

```
    public static void main(String[] args) {
        BankAccount user = new BankAccountProxy("User");
        user.accessAccount(); // denied

        BankAccount admin = new BankAccountProxy("Admin");
        admin.accessAccount(); // granted
    }
}
```

This way, Proxy not only saves cost and resources but also strengthens system design by separating responsibilities.

13. What is the Strategy Design Pattern in Java?

Answer

Strategy is about defining a family of algorithms and making them interchangeable so the client can choose the behavior at runtime. Instead of hardcoding logic, you pass in a strategy object that implements the behavior.

Think of a shopping app where the user can pay with credit card, UPI, or PayPal. The checkout flow shouldn't care about the details of payment—only which strategy is plugged in.

Example:

```
// Strategy
interface PaymentStrategy { void pay(int amount); }

// Concrete strategies
class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid " + amount + " via Credit Card");
    }
}
class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid " + amount + " via PayPal");
    }
}

// Context
class ShoppingCart {
    private PaymentStrategy strategy;
    public ShoppingCart(PaymentStrategy strategy) { this.strategy = strategy; }
    public void checkout(int amount) { strategy.pay(amount); }
}

// usage
public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart(new CreditCardPayment());
        cart.checkout(500);
        cart = new ShoppingCart(new PayPalPayment());
        cart.checkout(1200);
    }
}
```

This way, adding UPI or Wallet support later doesn't touch existing cart code.

14. What are the advantages of the Strategy Design Pattern?

Answer

1. Open/Closed Principle

   You can add a new algorithm by writing a new strategy class, no need to edit existing logic.
2. Encapsulation

   Each algorithm is isolated in its own class, which keeps code focused and easier to test.
3. Runtime flexibility

   The client can switch strategies dynamically—say, choosing “Express Delivery” vs. “Standard Delivery” in an e-commerce checkout.
4. Reusability

   Strategies can be reused across multiple contexts. A MergeSort strategy, for example, could be used in both analytics and reporting modules.

Example with sorting:

```
interface SortingStrategy { void sort(int[] arr); }

class BubbleSort implements SortingStrategy {
    public void sort(int[] arr) {
        System.out.println("Sorting with BubbleSort");
        // bubble sort logic
    }
}
class MergeSort implements SortingStrategy {
    public void sort(int[] arr) {
        System.out.println("Sorting with MergeSort");
        // merge sort logic
    }
}

public class Main {
    public static void main(String[] args) {
        SortingStrategy sorter = new BubbleSort();
        sorter.sort(new int[]{5, 3, 8});
        sorter = new MergeSort();
        sorter.sort(new int[]{5, 3, 8});
    }
}
```

This makes the code more adaptable to change, which is key in fast-moving domains like payments, pricing, or logistics.

15. What is the Observer Design Pattern in Java?

Answer

Observer establishes a one-to-many relationship where when one object changes state, all its dependents are notified automatically. It's heavily used in event-driven systems and GUIs.

A real-world analogy: You subscribe to a YouTube channel. Whenever the creator uploads a video, YouTube notifies all subscribers.

Example with a newsletter system:

```
// Observer
interface Observer { void update(String msg); }

// Concrete observer
class EmailSubscriber implements Observer {
    private String name;
    EmailSubscriber(String name) { this.name = name; }
    public void update(String msg) {
        System.out.println(name + " received: " + msg);
    }
}

// Subject
interface Subject {
    void subscribe(Observer obs);
    void unsubscribe(Observer obs);
    void notifyObservers();
}

// Concrete subject
class Newsletter implements Subject {
    private List<Observer> subscribers = new ArrayList<>();
    private String message;

    public void setMessage(String message) {
        this.message = message;
        notifyObservers();
    }
    public void subscribe(Observer obs) { subscribers.add(obs); }
    public void unsubscribe(Observer obs) { subscribers.remove(obs); }
    public void notifyObservers() {
        for (Observer obs : subscribers) obs.update(message);
    }
}
```

```
// usage
public class Main {
    public static void main(String[] args) {
        Newsletter newsletter = new Newsletter();
        Observer u1 = new EmailSubscriber("User1");
        Observer u2 = new EmailSubscriber("User2");
        newsletter.subscribe(u1);
        newsletter.subscribe(u2);
        newsletter.setMessage("New Edition Released!");
    }
}
```

This pattern is very common in Java frameworks—like PropertyChangeListener in Swing, or even the way Kafka consumers subscribe to topics.

16. What are the benefits of the Observer Design Pattern?

Answer

The Observer pattern shines in scenarios where multiple objects need to react automatically to changes in another object. Its main benefits are:

1. Loose coupling
   The subject only knows it has a list of observers—it doesn't need to know their concrete types. This makes both sides independently extensible.
2. Automatic updates
   Whenever the subject changes state, all observers get notified immediately. No manual polling is required.
3. Dynamic behavior
   Observers can be subscribed or unsubscribed at runtime, giving flexibility for scenarios like live dashboards or event-driven UIs.
4. Reusability
   Common subject–observer logic (subscribe, unsubscribe, notify) is abstracted and reusable across many domains—stock tracking, news feeds, chat applications, etc.

Example: stock price tracker
If a stock's price changes, all subscribed investors (observers) automatically get notified. You don't need to modify investor classes when adding a new type of alert (like SMS, email, or mobile push).

17. What is the Command Design Pattern in Java?

Answer

Command encapsulates a request into an object, so you can parameterize invokers with different commands, queue them, log them, and even support undo/redo. The key idea is decoupling the invoker (who triggers an action) from the receiver (who performs the action).

Analogy: Think of a waiter in a restaurant. The waiter takes an order (command) and passes it to the kitchen (receiver). The waiter doesn't cook the food—they just invoke the command.

Example with a smart home light:

```
// Command
interface Command { void execute(); }

// Receiver
class Light {
    void turnOn() { System.out.println("Light is ON"); }
    void turnOff() { System.out.println("Light is OFF"); }
}

// Concrete commands
class TurnOnCommand implements Command {
    private Light light;
    TurnOnCommand(Light light) { this.light = light; }
    public void execute() { light.turnOn(); }
}
class TurnOffCommand implements Command {
    private Light light;
    TurnOffCommand(Light light) { this.light = light; }
    public void execute() { light.turnOff(); }
}

// Invoker
class RemoteControl {
    private Command command;
    void setCommand(Command command) { this.command = command; }
    void pressButton() { command.execute(); }
}

// Usage
public class Main {
    public static void main(String[] args) {
```

```
        Light light = new Light();
        RemoteControl remote = new RemoteControl();

        remote.setCommand(new TurnOnCommand(light));
        remote.pressButton();

        remote.setCommand(new TurnOffCommand(light));
        remote.pressButton();
    }
}
```

This makes it very easy to extend the system later with new commands like DimLightCommand without modifying existing code.

18. What are the benefits of the Command Design Pattern?

Answer

1. Encapsulation of requests

   Each action is a command object, so logic and data needed to perform an operation stay together.
2. Undo/Redo support

   Since commands are objects, you can keep a history and reverse actions—very useful in text editors, IDEs, or transaction systems.
3. Extensibility

   New commands can be added with minimal changes. The invoker doesn't need to know details; it just executes the command.
4. Decoupling

   Invoker and receiver don't interact directly. The invoker only knows about the command interface, which increases flexibility.

Example with a text editor:

```
class TextEditor {
    void copy() { System.out.println("Text Copied"); }
    void paste() { System.out.println("Text Pasted"); }
}

// Commands
class CopyCommand implements Command {
    private TextEditor editor;
    CopyCommand(TextEditor editor) { this.editor = editor; }
    public void execute() { editor.copy(); }
}
class PasteCommand implements Command {
    private TextEditor editor;
    PasteCommand(TextEditor editor) { this.editor = editor; }
    public void execute() { editor.paste(); }
}

// Usage
public class Main {
    public static void main(String[] args) {
        TextEditor editor = new TextEditor();
        RemoteControl remote = new RemoteControl();

        remote.setCommand(new CopyCommand(editor));
```

```
        remote.pressButton();

        remote.setCommand(new PasteCommand(editor));
        remote.pressButton();
    }
}
```

This design makes it trivial to extend the editor with more commands like Cut, Undo, or Redo, simply by adding new command classes.

19. What is the Model-View-Controller (MVC) Pattern in Java?

Answer

MVC is an architectural pattern that separates an application into three parts:

1. Model → represents the data and business logic (like database entities, rules, or services).
2. View → handles how the data is displayed to the user (UI layer).
3. Controller → takes input from the user, updates the Model, and decides which View to show.

This separation improves organization and makes it easier to maintain or test.

Example with a student system:

```
// Model
class Student {
    private String name;
    private int rollNo;

    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getRollNo() { return rollNo; }
    public void setRollNo(int rollNo) { this.rollNo = rollNo; }
}

// View
class StudentView {
    public void showDetails(String name, int rollNo) {
        System.out.println("Student: " + name + " | Roll No: " + rollNo);
    }
}

// Controller
class StudentController {
    private Student model;
    private StudentView view;
    StudentController(Student model, StudentView view) {
        this.model = model; this.view = view;
    }
    public void setName(String name) { model.setName(name); }
```

```
    public void updateView() { view.showDetails(model.getName(),
model.getRollNo()); }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Student model = new Student("John", 1);
        StudentView view = new StudentView();
        StudentController controller = new StudentController(model, view);

        controller.updateView();
        controller.setName("Alex");
        controller.updateView();
    }
}
```

Frameworks like Spring MVC use this same concept at a much larger scale—controllers handle HTTP requests, models represent domain data, and views are JSP/Thymeleaf/JSON responses.

20. What are the advantages of the MVC Pattern?

Answer

1. Separation of concerns

   Each part has a single responsibility—UI changes don't affect the business logic.
2. Modularity

   You can swap or extend one part (say, change the view layer from JSP to Angular) without rewriting everything.
3. Reusability

   The same Model can be reused with different Views. For example, a Book model in a library app could be shown in an admin dashboard, a public site, or an API response.
4. Testability

   Since logic is separated, Models and Controllers can be tested independently without worrying about UI.

Practical example: In a library management system, the Model manages books, students, and transactions. The View could be a web UI or even a mobile app. The Controller takes user input like "borrow book" or "return book" and applies the right logic in the Model.

21. How does Dependency Injection (DI) relate to the Dependency Inversion Principle in Java?

Answer

Dependency Inversion is one of the SOLID principles. It says:

- High-level modules (business logic) should not depend on low-level modules (implementations).
- Both should depend on abstractions.

Dependency Injection is the way we achieve this in practice. Instead of a class creating its dependencies directly, they're "injected" from outside (via constructor, setter, or framework). This keeps code flexible, testable, and loosely coupled.

Example: message service system.

```
// Abstraction
interface MessageService {
    void sendMessage(String msg, String recipient);
}

// Implementation
class EmailService implements MessageService {
    public void sendMessage(String msg, String recipient) {
        System.out.println("Email to " + recipient + ": " + msg);
    }
}

// Consumer
class MessageProcessor {
    private MessageService service;
    // constructor injection
    MessageProcessor(MessageService service) { this.service = service; }
    public void process(String msg, String recipient) {
        service.sendMessage(msg, recipient);
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        MessageService service = new EmailService();
        MessageProcessor processor = new MessageProcessor(service);
        processor.process("Hello", "abc@gmail.com");
    }
```

```
}
```

Here, MessageProcessor doesn’t care if the service is email, SMS, or WhatsApp. It depends on the abstraction, not the detail. Frameworks like Spring take this further by automatically injecting dependencies through annotations like @Autowired.

22. What are the different types of Dependency Injection in Java?

Answer

There are three common ways to implement Dependency Injection in Java:

1. Constructor Injection

   Dependencies are passed through the constructor. This is the most recommended style because it ensures that the dependency is always available when the object is created. It also makes the class immutable if fields are declared final.

```
class Service {}
class Client {
    private final Service service;
    public Client(Service service) { this.service = service; }
}
```

2. Setter Injection

   Dependencies are provided through public setter methods. This is more flexible if dependencies are optional or can change after object creation, but it makes the object mutable.

```
class Client {
    private Service service;
    public void setService(Service service) { this.service = service; }
}
```

3. Field Injection

   Dependencies are directly injected into fields using reflection (typically by frameworks like Spring). While concise, it reduces testability since dependencies aren't explicit in constructors.

```
@Component
class Client {
```

```
    @Autowired
    private Service service;
}
```

In practice:

- Constructor Injection is preferred for mandatory dependencies.
- Setter Injection is useful for optional or changeable ones.
- Field Injection is convenient but best kept for small cases, as it hides the true dependency contract.

23. What is the difference between Singleton Pattern and Dependency Injection?

Answer

They both deal with object creation, but their goals are different:

1. Singleton Pattern

- Ensures only one instance of a class exists.
- Used for global, shared resources like logging or configuration.
- Enforces control over instantiation inside the class.

```
class Singleton {
    private static Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

2. Dependency Injection

- Delegates object creation and binding to an external container or framework.
- Encourages loose coupling by injecting abstractions at runtime.
- Adheres to the Dependency Inversion Principle by inverting control.

```
class Client {
    private final Service service;
    public Client(Service service) { this.service = service; }
}
```

Key difference: Singleton is about limiting object count, while DI is about flexibility and testability. For example, in Spring you can combine both by declaring a bean as a singleton scope, but the creation and wiring are handled by the container, not hardcoded in the class.

24. What are the real-world use cases of the Strategy Design Pattern?

Answer

Strategy is valuable wherever multiple approaches exist for the same task, and you want to switch between them dynamically. Some use cases:

1. Payment systems

   Choosing between credit card, UPI, or PayPal during checkout.
2. Sorting systems

   Switching between MergeSort or QuickSort depending on dataset size.
3. Logging frameworks

   Selecting whether logs go to console, file, or a remote server.
4. Compression utilities

   Allowing ZIP, RAR, or GZIP compression strategies for files.

Example with compression strategies:

```
interface CompressionStrategy { void compress(String data); }

class ZipCompression implements CompressionStrategy {
    public void compress(String data) {
        System.out.println("Compressing with ZIP");
    }
}
class RarCompression implements CompressionStrategy {
    public void compress(String data) {
        System.out.println("Compressing with RAR");
    }
}

class FileCompressor {
    private CompressionStrategy strategy;
    FileCompressor(CompressionStrategy strategy) { this.strategy = strategy; }
    public void compress(String data) { strategy.compress(data); }
}

// usage
public class Main {
    public static void main(String[] args) {
        FileCompressor compressor = new FileCompressor(new ZipCompression());
        compressor.compress("file data");
        compressor = new FileCompressor(new RarCompression());
        compressor.compress("file data");
```

```
    }
}
```

This pattern helps make systems extensible and avoids cluttering the code with conditionals.

25. How is the Observer Pattern used in Event Handling Systems?

Answer

The Observer pattern is the backbone of event-driven programming. Whenever an event occurs (like a button click, a stock price change, or a new message in a chat), the subject automatically notifies all observers that subscribed to it. This is how GUIs, message queues, and publish–subscribe systems operate.

In Java, traditional event handling uses this approach: the source (subject) fires events, and registered listeners (observers) respond.

Example with Java's built-in Observable and Observer (legacy, but simple for demonstration):

```
import java.util.Observable;
import java.util.Observer;

// Subject
class EventSource extends Observable {
    void triggerEvent(String msg) {
        setChanged();
        notifyObservers(msg); // notify all observers
    }
}

// Observer
class EventObserver implements Observer {
    private String name;
    EventObserver(String name) { this.name = name; }
    @Override
    public void update(Observable o, Object arg) {
        System.out.println(name + " received: " + arg);
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        EventSource source = new EventSource();
        Observer obs1 = new EventObserver("Observer 1");
        Observer obs2 = new EventObserver("Observer 2");

        source.addObserver(obs1);
        source.addObserver(obs2);
```

```
        source.triggerEvent("Event Triggered!");
    }
}
```

In modern frameworks like Swing, JavaFX, or Spring, the same concept is used but with custom listener interfaces or annotations. For example, a Swing button click event is just the Observer pattern under the hood.

26. What is Lazy Initialization in Singleton? How is it implemented?

Answer

Lazy initialization means delaying the creation of a Singleton object until it's actually needed. If the instance is never requested, it's never created—saving resources.

Without lazy initialization, the Singleton instance is created at class loading time (eager initialization).

The standard approach is using double-checked locking:

```
class LazySingleton {
    private static volatile LazySingleton instance; // volatile for thread safety
    private LazySingleton() {}

    public static LazySingleton getInstance() {
        if (instance == null) { // first check
            synchronized (LazySingleton.class) {
                if (instance == null) { // second check
                    instance = new LazySingleton();
                }
            }
        }
        return instance;
    }
}
```

Other approaches include:

- Using the Initialization-on-demand holder idiom (best for simplicity and thread-safety).
- Using an enum for safe and simple Singleton.

Lazy initialization is especially useful when object creation is expensive (e.g., a connection pool or a configuration loader).

27. How does the Decorator Pattern compare to Subclassing?

Answer

Both are ways to extend functionality, but they work very differently:

1. Decorator Pattern

- Extends behavior dynamically at runtime.
- Uses composition (wrapping objects) instead of inheritance.
- Multiple decorators can be stacked together.
- More flexible because you don’t need to create a new subclass for every variation.

2. Subclassing

- Extends behavior at compile-time by creating new subclasses.
- Each variation requires a new class, which leads to class explosion.
- Less flexible because behaviors are fixed once compiled.

Example with cars:

```
// Component
interface Car { String assemble(); }

// Concrete component
class BasicCar implements Car {
    public String assemble() { return "Basic Car"; }
}

// Decorator
class SportsCarDecorator implements Car {
    private Car car;
    SportsCarDecorator(Car car) { this.car = car; }
    public String assemble() { return car.assemble() + " with Sports Features"; }
}

// usage
public class Main {
    public static void main(String[] args) {
        Car sportsCar = new SportsCarDecorator(new BasicCar());
        System.out.println(sportsCar.assemble()); // Basic Car with Sports Features
    }
}
```

If you used subclassing, you'd need SportsCar, LuxuryCar, SportsLuxuryCar, etc.—which grows rapidly. With decorators, you can compose features at runtime, making it much cleaner and more maintainable.

28. How is MVC used in Web Applications?

Answer

MVC in web applications separates the backend logic, UI, and request handling so teams can work independently and code stays clean.

1. Model → contains data, validation, and business rules (e.g., Java entities, service classes).
2. View → represents the UI layer (HTML, CSS, JSP, Thymeleaf templates, or JSON responses).
3. Controller → maps HTTP requests to business logic and decides which view to render.

Example with Spring MVC:

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String home(Model model) {
        model.addAttribute("message", "Welcome to MVC!");
        return "home"; // mapped to home.jsp or home.html
    }
}
```

Here:

- Model carries the data (the message).
- View (JSP/Thymeleaf template) renders the UI.
- Controller handles the request and coordinates between Model and View.

This separation makes it easy to swap a UI layer (say, HTML to JSON API) without touching business logic.

29. How does the Proxy Pattern improve performance in large systems?

Answer

The Proxy pattern helps optimize performance by acting as a lightweight placeholder for heavy or remote resources, loading them only when required. This is known as a Virtual Proxy.

Use cases:

- Lazy loading large images or documents in a content management system.
- Deferring database connections until data is actually accessed.
- Acting as a cache proxy to avoid hitting remote services repeatedly.

Example:

```
interface Image { void display(); }

class RealImage implements Image {
    private String fileName;
    RealImage(String fileName) {
        this.fileName = fileName;
        loadFromDisk(); // expensive operation
    }
    private void loadFromDisk() { System.out.println("Loading " + fileName); }
    public void display() { System.out.println("Displaying " + fileName); }
}

class ImageProxy implements Image {
    private RealImage realImage;
    private String fileName;
    ImageProxy(String fileName) { this.fileName = fileName; }
    public void display() {
        if (realImage == null) { realImage = new RealImage(fileName); }
        realImage.display();
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        Image image = new ImageProxy("bigphoto.jpg");
        image.display(); // loads + displays
        image.display(); // only displays, no reload
    }
```

```
}
```

This approach improves responsiveness, especially in large-scale systems dealing with expensive resources.

30. What is the role of DI in Testing?

Answer

Dependency Injection makes testing easier by allowing you to replace real dependencies with mocks or stubs. This isolates the unit under test, making tests faster and more predictable.

Without DI, a service might directly create its repository, forcing tests to hit a real database. With DI, you inject a mock repository.

Example:

```
interface Repository {
    String fetchData();
}

class Service {
    private Repository repo;
    public Service(Repository repo) { this.repo = repo; }
    public String getData() { return repo.fetchData(); }
}

// Mock dependency
class MockRepository implements Repository {
    public String fetchData() { return "Mock Data"; }
}

// usage in test
public class TestService {
    public static void main(String[] args) {
        Repository mockRepo = new MockRepository();
        Service service = new Service(mockRepo);
        System.out.println(service.getData()); // Mock Data
    }
}
```

This ensures tests don’t depend on network, databases, or external systems. Frameworks like Mockito and Spring Boot Test build on this principle, injecting mocks automatically.

31. What is the difference between Factory Method and Abstract Factory Design Pattern in Java?

Answer

Both are creational patterns but used at different scales.

1. Factory Method

- Defines a method that subclasses override to decide which object to create.
- Produces a single product at a time.
- Relies on inheritance.

Example:

```
// Factory Method
abstract class ShapeFactory {
    public abstract Shape createShape();
}

class CircleFactory extends ShapeFactory {
    public Shape createShape() { return new Circle(); }
}

interface Shape { void draw(); }
class Circle implements Shape {
    public void draw() { System.out.println("Drawing Circle"); }
}

// usage
public class Main {
    public static void main(String[] args) {
        ShapeFactory factory = new CircleFactory();
        Shape shape = factory.createShape();
        shape.draw();
    }
}
```

2. Abstract Factory

- Creates families of related products without specifying concrete classes.
- Groups multiple factory methods together in one factory interface.
- Encapsulates object creation for a whole product family.

Example with GUI components:

```
// Abstract Factory
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

// Concrete Factory
class MacGUIFactory implements GUIFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}

// Products
interface Button { void render(); }
class MacButton implements Button {
    public void render() { System.out.println("Mac Button"); }
}
interface Checkbox { void render(); }
class MacCheckbox implements Checkbox {
    public void render() { System.out.println("Mac Checkbox"); }
}

// usage
public class Main {
    public static void main(String[] args) {
        GUIFactory factory = new MacGUIFactory();
        factory.createButton().render();
        factory.createCheckbox().render();
    }
}
```

In short:

- Factory Method → one product at a time, subclasses decide creation.
- Abstract Factory → a family of related products, grouped under one factory.

32. How does the Command Design Pattern handle Undo and Redo operations?

Answer

Command objects can store the state of an operation and provide both execute() and undo() methods. By keeping a history of commands in a stack, you can easily roll back (undo) or reapply (redo) operations.

1. Undo → pop the last executed command from the stack and call undo().
2. Redo → push the command back and call execute() again.

Example with a Light:

```
interface Command {
    void execute();
    void undo();
}

// Receiver
class Light {
    void turnOn() { System.out.println("Light ON"); }
    void turnOff() { System.out.println("Light OFF"); }
}

// Concrete Command
class LightOnCommand implements Command {
    private Light light;
    LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.turnOn(); }
    public void undo() { light.turnOff(); }
}

// Invoker
class CommandInvoker {
    private Stack<Command> history = new Stack<>();

    public void executeCommand(Command cmd) {
        cmd.execute();
        history.push(cmd);
    }
    public void undoLast() {
        if (!history.isEmpty()) history.pop().undo();
    }
}

// usage
```

```
public class Main {
    public static void main(String[] args) {
        Light light = new Light();
        CommandInvoker invoker = new CommandInvoker();

        Command onCommand = new LightOnCommand(light);
        invoker.executeCommand(onCommand); // Light ON
        invoker.undoLast();                // Light OFF
    }
}
```

This pattern is widely used in text editors, IDEs, or transactional systems where undo/redo is required.

33. What are the advantages and disadvantages of the Observer Pattern?

Answer

Observer has clear strengths but also some trade-offs, especially in large systems.

Advantages

1. Loose coupling → The subject doesn't need to know the details of its observers, just that they implement the observer interface. This makes both sides independently extensible.
2. Automatic updates → Observers always get notified when the subject changes, removing the need for manual polling.
3. Dynamic subscriptions → Observers can be added or removed at runtime, which gives flexibility in systems like chat apps or stock trackers.

Disadvantages

1. Performance issues → If there are many observers, a single update can trigger a heavy cascade of notifications.
2. Debugging complexity → Since communication is indirect, tracing who triggered what can be difficult.
3. Risk of memory leaks → If observers are not unsubscribed properly, they may hold unnecessary references and prevent garbage collection.

Example: A stock price monitoring system where all subscribed clients (mobile apps, dashboards, trading bots) automatically get updates when a stock price changes. This is efficient but could become slow if thousands of clients subscribe to a single stock.

34. How can the Decorator Pattern improve logging functionality?

Answer

Decorator allows you to extend logging behavior dynamically without changing the base logger class. You can wrap the logger with decorators that add features like timestamps, severity levels, or even saving logs to a database.

Example with a timestamp decorator:

```
interface Logger {
    void log(String msg);
}

class ConsoleLogger implements Logger {
    public void log(String msg) { System.out.println(msg); }
}

// Decorator
class TimestampLoggerDecorator implements Logger {
    private Logger logger;
    TimestampLoggerDecorator(Logger logger) { this.logger = logger; }
    public void log(String msg) {
        logger.log("[Timestamp: " + System.currentTimeMillis() + "] " + msg);
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        Logger logger = new TimestampLoggerDecorator(new ConsoleLogger());
        logger.log("Application started");
    }
}
```

The beauty is that you can stack multiple decorators: one for timestamps, another for severity level, another for saving to a file—without modifying the base ConsoleLogger.

35. Explain the role of MVC in a Spring Framework-based Web Application.

Answer

Spring MVC follows the MVC pattern to build clean, testable web applications:

1. Model → represents data and business logic. Usually service classes, repositories (DAOs), and entities (e.g., JPA).
2. View → UI layer rendered as JSP, Thymeleaf, or JSON responses in case of REST APIs.
3. Controller → maps HTTP requests to service methods, prepares data, and decides which view to render.

Example with an employee management app:

```
@Controller
public class EmployeeController {

    @Autowired
    private EmployeeService employeeService;

    @GetMapping("/employees")
    public String getEmployees(Model model) {
        List<Employee> employees = employeeService.getAllEmployees();
        model.addAttribute("employees", employees);
        return "employeeList"; // resolves to Thymeleaf template
    }
}
```

Here:

- The Controller (EmployeeController) handles /employees requests.
- The Model (EmployeeService + Employee entity) fetches data.
- The View (employeeList.html) displays the list of employees.

This separation makes the app modular: the UI can be swapped (JSP → React frontend) without touching backend services, and business logic can be tested independently of the view.

36. What are some real-world scenarios for the Proxy Design Pattern?

Answer

The Proxy pattern is very practical when working with expensive or sensitive resources. Some common real-world uses:

1. Virtual Proxy → Delays the creation of resource-heavy objects until they are really needed. For example, loading large product images in an e-commerce catalog only when the user clicks on them.
2. Protection Proxy → Controls access to sensitive operations based on permissions. For example, in a banking system, only admins can access account details.
3. Remote Proxy → Represents an object located in another JVM, server, or microservice. RMI (Remote Method Invocation) in Java is an example.
4. Caching Proxy → Stores results of expensive calls and serves them quickly on repeated requests. This is common in APIs or microservices to reduce database or network load.

Example with caching proxy:

```
interface DataService { String fetchData(); }

class RealDataService implements DataService {
    public String fetchData() {
        System.out.println("Fetching from DB...");
        return "Real Data";
    }
}

// Proxy with caching
class DataProxy implements DataService {
    private RealDataService realService;
    private String cache;

    public String fetchData() {
        if (cache == null) {
            realService = new RealDataService();
            cache = realService.fetchData();
        }
        return cache; // returns cached data next time
    }
```

```
}

// usage
public class Main {
    public static void main(String[] args) {
        DataService service = new DataProxy();
        System.out.println(service.fetchData()); // fetches real
        System.out.println(service.fetchData()); // returns cached
    }
}
```

This is exactly how reverse proxies or API gateways work in large-scale distributed systems.

37. How does the Builder Pattern simplify object construction in complex scenarios?

Answer

Builder makes object creation cleaner when the object has many attributes, some optional, some mandatory. Instead of constructors with many arguments (the telescoping constructor problem), Builder lets you set fields step by step.

Real-world example: a hotel booking system, where each booking can have different combinations like room type, meals, airport pickup, etc.

```
class Booking {
    private String roomType;
    private boolean breakfastIncluded;

    private Booking(Builder b) {
        this.roomType = b.roomType;
        this.breakfastIncluded = b.breakfastIncluded;
    }

    static class Builder {
        private String roomType;
        private boolean breakfastIncluded;

        public Builder setRoomType(String roomType) {
            this.roomType = roomType; return this;
        }
        public Builder includeBreakfast(boolean include) {
            this.breakfastIncluded = include; return this;
        }
        public Booking build() { return new Booking(this); }
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        Booking booking = new Booking.Builder()
            .setRoomType("Deluxe")
            .includeBreakfast(true)
            .build();

        System.out.println("Booking confirmed for " + booking);
    }
}
```

This approach improves readability, keeps objects immutable, and allows clear configuration for complex scenarios.

38. How can the Strategy Pattern be used in AI decision-making systems?

Answer

In AI systems, Strategy is very useful because the system often needs to adapt its behavior dynamically based on the situation. Each decision-making algorithm can be modeled as a strategy and swapped at runtime.

Example in a chess AI:

```
interface Strategy { void execute(); }

class AggressiveStrategy implements Strategy {
    public void execute() {
        System.out.println("Playing aggressively...");
    }
}
class DefensiveStrategy implements Strategy {
    public void execute() {
        System.out.println("Playing defensively...");
    }
}

class ChessAI {
    private Strategy strategy;
    public void setStrategy(Strategy strategy) { this.strategy = strategy; }
    public void play() { strategy.execute(); }
}

// usage
public class Main {
    public static void main(String[] args) {
        ChessAI ai = new ChessAI();
        ai.setStrategy(new AggressiveStrategy());
        ai.play(); // AI plays aggressively

        ai.setStrategy(new DefensiveStrategy());
        ai.play(); // AI switches to defense
    }
}
```

This is the same principle used in game AI, recommendation engines, or even self-driving cars—where the system chooses the appropriate algorithm (strategy) based on current environment or state.

39. How does the Observer Pattern handle real-time notifications in event-driven systems?

Answer

In event-driven systems, the Observer pattern ensures that when an event occurs (subject state changes), all subscribed observers are notified immediately. This makes it ideal for real-time use cases like:

- Stock trading platforms → clients get notified instantly when stock prices change.
- Social media → followers are alerted when someone posts new content.
- Chat applications → users receive real-time message notifications.

The subject keeps a list of observers and, on state change, broadcasts the update. Observers react independently, without the subject knowing their details.

Example: stock price tracker

```
// Subject
interface Stock {
    void subscribe(Observer o);
    void unsubscribe(Observer o);
    void notifyObservers();
}

class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private double price;

    public void setPrice(double price) {
        this.price = price;
        notifyObservers();
    }

    public void subscribe(Observer o) { observers.add(o); }
    public void unsubscribe(Observer o) { observers.remove(o); }
    public void notifyObservers() {
        for (Observer o : observers) {
            o.update(price);
        }
    }
}

// Observer
```

```
interface Observer { void update(double price); }

class Trader implements Observer {
    private String name;
    Trader(String name) { this.name = name; }
    public void update(double price) {
        System.out.println(name + " notified: Stock price is now " + price);
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        StockMarket stock = new StockMarket();
        Observer t1 = new Trader("Alice");
        Observer t2 = new Trader("Bob");

        stock.subscribe(t1);
        stock.subscribe(t2);

        stock.setPrice(101.5); // Notifies Alice & Bob
        stock.setPrice(105.0); // Real-time notification again
    }
}
```

This mechanism ensures real-time flow without tight coupling. Frameworks like Kafka or RabbitMQ extend this concept at scale.

40. What is Dependency Injection in Spring, and how is it implemented?

Answer

Dependency Injection (DI) in Spring is how the framework manages object creation and wiring. Instead of classes instantiating their dependencies manually, Spring injects them automatically. This ensures loose coupling, easy testing, and centralized configuration.

Spring supports multiple ways to inject dependencies:

1. **Annotation-based (@Autowired)** → the most common, where Spring scans and wires beans automatically.
2. **Java-based configuration (@Bean)** → explicit bean definitions in a @Configuration class.
3. **XML configuration** → the older approach, less common in modern Spring apps.

Example with @Autowired constructor injection:

```
@Service
public class EmployeeService {
    private final EmployeeRepository repository;

    @Autowired
    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }
}

@Repository
public class EmployeeRepository {
    public List<Employee> findAll() {
        return List.of(new Employee("John"), new Employee("Alex"));
    }
}
```

Spring container detects @Service and @Repository during component scanning, creates beans, and injects EmployeeRepository into EmployeeService.

This approach makes testing easier too—you can inject a mock repository instead of the real one.

41. What is the Prototype Design Pattern in Java, and when would you use it?

Answer

Prototype is a creational pattern where new objects are created by cloning an existing object (prototype), rather than instantiating a new one from scratch.

It's useful when:

- Object creation is expensive (e.g., parsing, DB fetch, network call).
- You need multiple copies of objects with slight modifications.

In Java, this is done with the clone() method or copy constructors.

Example:

```
class Document implements Cloneable {
    private String content;
    Document(String content) { this.content = content; }
    public Document clone() {
        try { return (Document) super.clone(); }
        catch (CloneNotSupportedException e) { throw new RuntimeException(e); }
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        Document d1 = new Document("Report");
        Document d2 = d1.clone(); // cheaper copy
        System.out.println("Cloned document created.");
    }
}
```

Real-world analogy: Instead of writing a new resume from scratch, you copy your old resume and tweak details.

42. What is the Object Pool Pattern, and how is it useful for resource management?

Answer

The Object Pool pattern manages a set of reusable objects, handing them out to clients instead of creating and destroying them repeatedly.

This is very useful for expensive resources like database connections, threads, or sockets. Java frameworks like JDBC use connection pools internally.

Example:

```
class ConnectionPool {
    private Queue<Connection> pool = new LinkedList<>();

    public ConnectionPool(int size) {
        for (int i = 0; i < size; i++) pool.add(new Connection("Conn-" + i));
    }

    public Connection acquire() { return pool.poll(); }
    public void release(Connection conn) { pool.offer(conn); }
}

class Connection {
    private String id;
    Connection(String id) { this.id = id; }
    public void use() { System.out.println("Using " + id); }
}
```

Instead of creating a new DB connection every time, apps reuse pooled connections, which improves performance and reduces resource exhaustion.

43. What is the Composite Pattern, and where is it used in Java frameworks?

Answer

Composite is a structural pattern that lets you treat individual objects and groups of objects uniformly.

It’s useful for representing part-whole hierarchies like files in directories, UI components in a window, or menu items in a menu.

Example with a file system:

```
interface FileSystem {
    void showDetails();
}

class File implements FileSystem {
    private String name;
    File(String name) { this.name = name; }
    public void showDetails() { System.out.println("File: " + name); }
}

class Directory implements FileSystem {
    private String name;
    private List<FileSystem> children = new ArrayList<>();
    Directory(String name) { this.name = name; }
    public void add(FileSystem fs) { children.add(fs); }
    public void showDetails() {
        System.out.println("Directory: " + name);
        for (FileSystem child : children) child.showDetails();
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        Directory root = new Directory("Root");
        root.add(new File("a.txt"));
        Directory sub = new Directory("SubDir");
        sub.add(new File("b.txt"));
        root.add(sub);
        root.showDetails();
    }
}
```

Frameworks:

- Swing/JavaFX → Component hierarchy for UI elements.
- DOM APIs → treat nodes and node groups uniformly.

44. What is the Facade Pattern, and how does it simplify subsystem usage?

Answer

The Facade pattern provides a simple, unified interface to a complex subsystem. Instead of exposing dozens of classes and methods, you give clients one entry point that internally coordinates all the work.

This is useful when dealing with complex APIs like payment gateways, multimedia libraries, or enterprise systems where clients only need a subset of functionality.

Example with a home theater system:

```
class DVDPlayer { void on() { System.out.println("DVD Player ON"); } }
class Projector { void on() { System.out.println("Projector ON"); } }
class SoundSystem { void on() { System.out.println("Surround Sound ON"); } }

class HomeTheaterFacade {
    private DVDPlayer dvd = new DVDPlayer();
    private Projector projector = new Projector();
    private SoundSystem sound = new SoundSystem();

    public void watchMovie() {
        dvd.on();
        projector.on();
        sound.on();
        System.out.println("Movie started!");
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        HomeTheaterFacade theater = new HomeTheaterFacade();
        theater.watchMovie();
    }
}
```

In Java, the java.util.logging.Logger is also a kind of facade—it hides internal complexity of handlers, filters, and formatters.

45. How does the Flyweight Pattern improve memory efficiency?

Answer

The Flyweight pattern reduces memory usage by sharing common, intrinsic state among many objects. Instead of creating thousands of identical objects, you reuse a single shared instance.

It’s especially useful in systems with a large number of similar objects, like game engines, text editors, or caching frameworks.

Example with characters in a text editor:

```
class Character {
    private char symbol;   // intrinsic (shared)
    private String font;   // intrinsic
    public Character(char symbol, String font) {
        this.symbol = symbol; this.font = font;
    }
    public void display(int size) { // extrinsic
        System.out.println(symbol + " in " + font + " size " + size);
    }
}

class CharacterFactory {
    private static final Map<String, Character> pool = new HashMap<>();
    public static Character get(char symbol, String font) {
        String key = symbol + "-" + font;
        return pool.computeIfAbsent(key, k -> new Character(symbol, font));
    }
}

// usage
public class Main {
    public static void main(String[] args) {
        Character c1 = CharacterFactory.get('A', "Arial");
        Character c2 = CharacterFactory.get('A', "Arial");
        System.out.println(c1 == c2); // true → reused
    }
}
```

Java examples:

- Integer.valueOf() caches small integers.
- String literals ("abc") are interned and reused from the String Pool.

46. What is the Bridge Pattern, and how does it differ from Adapter?

Answer

Bridge decouples an abstraction from its implementation so they can evolve independently. Instead of tying a class hierarchy to one implementation, you separate them into two dimensions and “bridge” them via composition.

This differs from Adapter:

- **Adapter** → makes incompatible interfaces work together (retrofitting).
- **Bridge** → is designed upfront to decouple abstraction from implementation.

Example with shapes and colors:

```
// Implementor
interface Color { String fill(); }
class Red implements Color { public String fill() { return "Red"; } }
class Blue implements Color { public String fill() { return "Blue"; } }

// Abstraction
abstract class Shape {
    protected Color color;
    Shape(Color color) { this.color = color; }
    abstract void draw();
}

// Refined Abstraction
class Circle extends Shape {
    Circle(Color color) { super(color); }
    void draw() { System.out.println("Drawing Circle in " + color.fill()); }
}

// usage
public class Main {
    public static void main(String[] args) {
        Shape redCircle = new Circle(new Red());
        redCircle.draw(); // Drawing Circle in Red
    }
}
```

Bridge is useful when you have two axes of variation. For example:

- Payment system → Payment type (CreditCard/UPI) vs. Region (US/India).
- UI toolkit → Shape (Circle/Rectangle) vs. Color (Red/Blue).

47. What is the Template Method Pattern in Java, and where is it used?

Answer

The Template Method pattern defines the skeleton of an algorithm in a base class but lets subclasses override specific steps without changing the overall flow.

It's useful when multiple classes share the same process but differ in small steps.

Example with data parsing:

```
abstract class DataParser {
    // template method
    public final void parse() {
        readData();
        processData();
        saveData();
    }
    abstract void readData();
    abstract void processData();
    void saveData() { System.out.println("Saving data to DB"); }
}

class CSVParser extends DataParser {
    void readData() { System.out.println("Reading CSV data"); }
    void processData() { System.out.println("Processing CSV data"); }
}

class XMLParser extends DataParser {
    void readData() { System.out.println("Reading XML data"); }
    void processData() { System.out.println("Processing XML data"); }
}

// usage
public class Main {
    public static void main(String[] args) {
        DataParser parser = new CSVParser();
        parser.parse(); // CSV steps + save
    }
}
```

Java example: java.util.AbstractList uses Template Method—subclasses implement specific behaviors but overall logic is controlled.

48. What is the State Pattern, and how does it model object behavior?

Answer

The State pattern allows an object to change its behavior when its internal state changes. Instead of using big if-else or switch statements, each state is represented by a class.

This is useful in scenarios like ATM machines, order processing, or workflow engines.

Example with an order lifecycle:

```
interface OrderState {
    void next(Order order);
    void prev(Order order);
    void printStatus();
}

class NewOrder implements OrderState {
    public void next(Order order) { order.setState(new ShippedOrder()); }
    public void prev(Order order) { System.out.println("Order is new, no previous
state."); }
    public void printStatus() { System.out.println("Order placed."); }
}

class ShippedOrder implements OrderState {
    public void next(Order order) { order.setState(new DeliveredOrder()); }
    public void prev(Order order) { order.setState(new NewOrder()); }
    public void printStatus() { System.out.println("Order shipped."); }
}

class DeliveredOrder implements OrderState {
    public void next(Order order) { System.out.println("Order already delivered."); }
    public void prev(Order order) { order.setState(new ShippedOrder()); }
    public void printStatus() { System.out.println("Order delivered."); }
}

class Order {
    private OrderState state = new NewOrder();
    public void setState(OrderState state) { this.state = state; }
    public void next() { state.next(this); }
    public void prev() { state.prev(this); }
    public void printStatus() { state.printStatus(); }
}
```

```
// usage
public class Main {
    public static void main(String[] args) {
        Order order = new Order();
        order.printStatus(); // placed
        order.next();        // move to shipped
        order.printStatus(); // shipped
    }
}
```

This keeps logic clean by pushing behavior into state classes.

49. What is the Mediator Pattern, and how does it simplify object communication?

Answer

Mediator centralizes communication between multiple objects so they don't talk to each other directly. Instead, they interact via the mediator, which reduces coupling.

This is useful in chat applications, UI frameworks, or air traffic control systems.

Example with a chatroom:

```
interface Mediator {
    void sendMessage(String msg, User user);
}

class ChatMediator implements Mediator {
    private List<User> users = new ArrayList<>();
    public void addUser(User user) { users.add(user); }
    public void sendMessage(String msg, User sender) {
        for (User u : users) {
            if (u != sender) u.receive(msg);
        }
    }
}

abstract class User {
    protected Mediator mediator;
    protected String name;
    User(Mediator mediator, String name) {
        this.mediator = mediator; this.name = name;
    }
    public abstract void send(String msg);
    public abstract void receive(String msg);
}

class ChatUser extends User {
    ChatUser(Mediator mediator, String name) { super(mediator, name); }
    public void send(String msg) { System.out.println(name + " sends: " + msg);
mediator.sendMessage(msg, this); }
    public void receive(String msg) { System.out.println(name + " received: " + msg);
}
}

// usage
public class Main {
    public static void main(String[] args) {
        ChatMediator mediator = new ChatMediator();
```

```
        User u1 = new ChatUser(mediator, "Alice");
        User u2 = new ChatUser(mediator, "Bob");
        mediator.addUser(u1);
        mediator.addUser(u2);

        u1.send("Hello everyone!"); // Bob receives
    }
}
```

Mediator avoids spaghetti communication between multiple objects by centralizing it.

50. What is the Chain of Responsibility Pattern, and where do we see it in Java?

Answer

Chain of Responsibility passes a request along a chain of handlers, where each handler decides either to process it or pass it to the next one.

It's useful for logging frameworks, request processing pipelines, and event handling.

Example: logging system with multiple levels.

```
abstract class Logger {
    public static int INFO = 1, ERROR = 2;
    protected int level;
    protected Logger nextLogger;

    public void setNext(Logger nextLogger) { this.nextLogger = nextLogger; }

    public void logMessage(int level, String message) {
        if (this.level <= level) write(message);
        if (nextLogger != null) nextLogger.logMessage(level, message);
    }

    protected abstract void write(String message);
}

class InfoLogger extends Logger {
    InfoLogger(int level) { this.level = level; }
    protected void write(String msg) { System.out.println("INFO: " + msg); }
}

class ErrorLogger extends Logger {
    ErrorLogger(int level) { this.level = level; }
    protected void write(String msg) { System.out.println("ERROR: " + msg); }
}

// usage
public class Main {
    public static void main(String[] args) {
        Logger error = new ErrorLogger(Logger.ERROR);
        Logger info = new InfoLogger(Logger.INFO);
        info.setNext(error);

        info.logMessage(Logger.INFO, "This is an info message");
        info.logMessage(Logger.ERROR, "This is an error message");
    }
}
```

Java example: Servlet filters (javax.servlet.Filter) are a chain of responsibility—each filter can handle or forward requests.

51. What is the Interpreter Pattern, and what are its real-world use cases?

Answer

Interpreter defines a representation for a language's grammar and provides an interpreter to evaluate sentences in the language.

It's useful for simple domain-specific languages (DSLs), query engines, expression parsing, or rule engines.

Example: evaluating expressions like "1 AND 0" or "1 OR 0".

```
interface Expression {
    boolean interpret();
}

class TerminalExpression implements Expression {
    private boolean value;
    TerminalExpression(boolean value) { this.value = value; }
    public boolean interpret() { return value; }
}

class AndExpression implements Expression {
    private Expression expr1, expr2;
    AndExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1; this.expr2 = expr2;
    }
    public boolean interpret() { return expr1.interpret() && expr2.interpret(); }
}

class OrExpression implements Expression {
    private Expression expr1, expr2;
    OrExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1; this.expr2 = expr2;
    }
    public boolean interpret() { return expr1.interpret() || expr2.interpret(); }
}

// usage
public class Main {
    public static void main(String[] args) {
        Expression expr = new OrExpression(new TerminalExpression(true),
                                           new TerminalExpression(false));
        System.out.println("Result: " + expr.interpret()); // true
    }
}
```

Real-world example:

- Regex engine in Java (java.util.regex.Pattern) is a form of interpreter.
- SQL parsing in ORMs.

52. What is the Visitor Pattern, and how does it help with extending operations?

Answer

Visitor lets you add new operations to existing object structures without modifying their classes. It separates algorithms from the objects on which they operate.

This is useful when you have a fixed set of object types but need to frequently add new operations.

Example: a document with different elements (Text, Image).

```
interface DocumentElement {
    void accept(Visitor visitor);
}

class TextElement implements DocumentElement {
    private String text;
    TextElement(String text) { this.text = text; }
    public void accept(Visitor visitor) { visitor.visit(this); }
    public String getText() { return text; }
}

class ImageElement implements DocumentElement {
    private String name;
    ImageElement(String name) { this.name = name; }
    public void accept(Visitor visitor) { visitor.visit(this); }
    public String getName() { return name; }
}

interface Visitor {
    void visit(TextElement text);
    void visit(ImageElement image);
}

class RenderVisitor implements Visitor {
    public void visit(TextElement text) { System.out.println("Rendering text: " +
text.getText()); }
    public void visit(ImageElement image) { System.out.println("Rendering image: "
+ image.getName()); }
}

// usage
public class Main {
    public static void main(String[] args) {
        List<DocumentElement> doc = List.of(new TextElement("Hello"), new
ImageElement("pic.png"));
```

```
        Visitor renderer = new RenderVisitor();
        doc.forEach(e -> e.accept(renderer));
    }
}
```

Java example: The javax.lang.model.element.Element API (used by annotation processors) uses Visitor pattern.

53. What is the Memento Pattern, and how does it enable undo functionality?

Answer

The Memento pattern captures and stores an object's state so it can be restored later, without exposing its internal details.

It's mainly used for **undo/redo functionality**, checkpoints, or state recovery.

Example: text editor with undo.

```
// Memento
class TextMemento {
    private final String state;
    TextMemento(String state) { this.state = state; }
    public String getState() { return state; }
}

// Originator
class TextEditor {
    private String text = "";
    public void type(String words) { text += words; }
    public String getText() { return text; }

    public TextMemento save() { return new TextMemento(text); }
    public void restore(TextMemento m) { this.text = m.getState(); }
}

// Caretaker
public class Main {
    public static void main(String[] args) {
        TextEditor editor = new TextEditor();
        editor.type("Hello ");
        TextMemento saved = editor.save(); // save checkpoint
        editor.type("World!");
        System.out.println(editor.getText()); // Hello World!

        editor.restore(saved); // undo
        System.out.println(editor.getText()); // Hello
    }
}
```

Java example: java.util.Date internally works like a memento for time state. In real apps, text editors, IDEs, and even database transactions use this concept.

54. What is the Iterator Pattern, and how is it implemented in Java Collections?

Answer

Iterator provides a way to access elements of a collection sequentially without exposing the underlying representation (array, list, set, etc.).

It encapsulates traversal logic so clients don’t need to know how elements are stored.

Example:

```
import java.util.Iterator;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> names = List.of("Alice", "Bob", "Charlie");
        Iterator<String> it = names.iterator();

        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}

Custom iterator example:


class NameRepository implements Iterable<String> {
    private String[] names = {"Tom", "Jerry", "Mickey"};
    public Iterator<String> iterator() {
        return new Iterator<>() {
            private int index = 0;
            public boolean hasNext() { return index < names.length; }
            public String next() { return names[index++]; }
        };
    }
}

// usage
public class Test {
    public static void main(String[] args) {
        for (String name : new NameRepository()) {
            System.out.println(name);
        }
    }
```

```
}
```

Java example: All Java Collections (List, Set, Map) implement Iterable and provide iterators. Enhanced for-loop (for-each) is just syntactic sugar over the Iterator pattern.